O que é um sistema generativo?

SGAIM — Semana 1

A promessa da semana passada

Na aula 0, disse-vos que a IA generativa é simples em princípio:

1. Observar dados
2. Estimar p(x)
3. Fazer sampling de novos dados

Mostrei um exemplo em 1D — alturas de pessoas.

Hoje vamos cumprir essa promessa — em 784 dimensões.

Mas primeiro: dois tipos de modelos

Um modelo **discriminativo** aprende uma fronteira.

Dada uma imagem x, pergunta: *o que é isto?*

Modela **p(y | x)** — a probabilidade de um label dado os dados.

Classificação, detecção, sentimento — tudo discriminativo.

A pergunta generativa é mais difícil

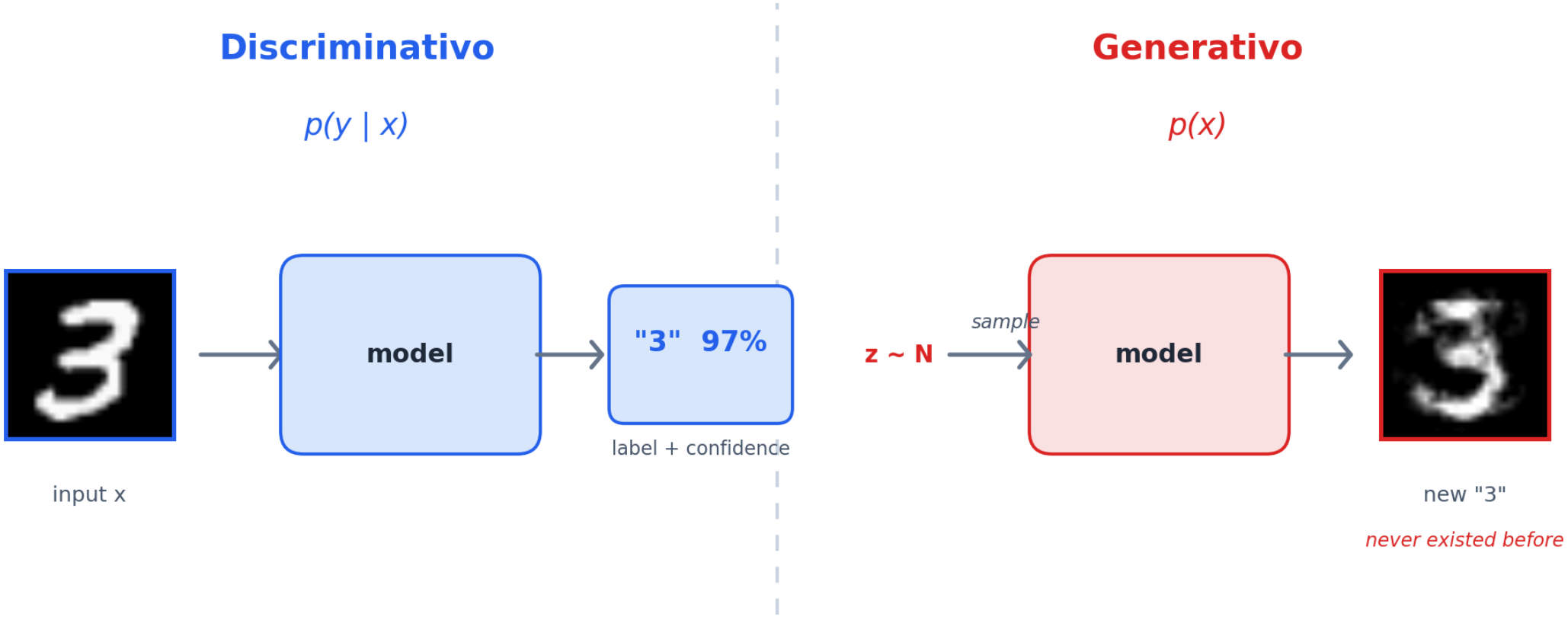
Um modelo **generativo** não classifica — **cria**.

Pergunta: *como é que o mundo se parece?*

Modela **p(x)** — a probabilidade dos dados em si.

Para gerar uma nova imagem, fazemos sampling de p(x).

Discriminativo vs generativo — lado a lado



Todos os sistemas de IA generativa — ChatGPT, DALL-E, Suno — modelam p(x).

O que é p(x), concretamente?

Em 1D: medir 1000 alturas → estimar a média μ, variância σ^2 → temos p(x).

Fazer sampling → obtemos uma altura plausível.

Isto É IA generativa. A ideia é idêntica a qualquer escala.

A única questão é: conseguimos fazer isto quando x não é um número, mas uma imagem — ou uma frase?

O caso 1D funciona lindamente

Alturas de adultos são aproximadamente Gaussianas.

Estimar $\mu \approx 170$ cm, $\sigma \approx 7$ cm.

Sampling → 168.3 cm. Plausível. 174.1 cm. Também plausível.

Uma dimensão. Uma bell curve. Fácil.

O que acontece quando passamos para 784 dimensões?

Agora: a maldição

Uma imagem grayscale 28×28 = um vector de 784 valores de pixel.

Cada pixel pode ter 256 valores (0–255).

Total de imagens possíveis: $\mathbf{256^{784} \approx 10^{1887}}$

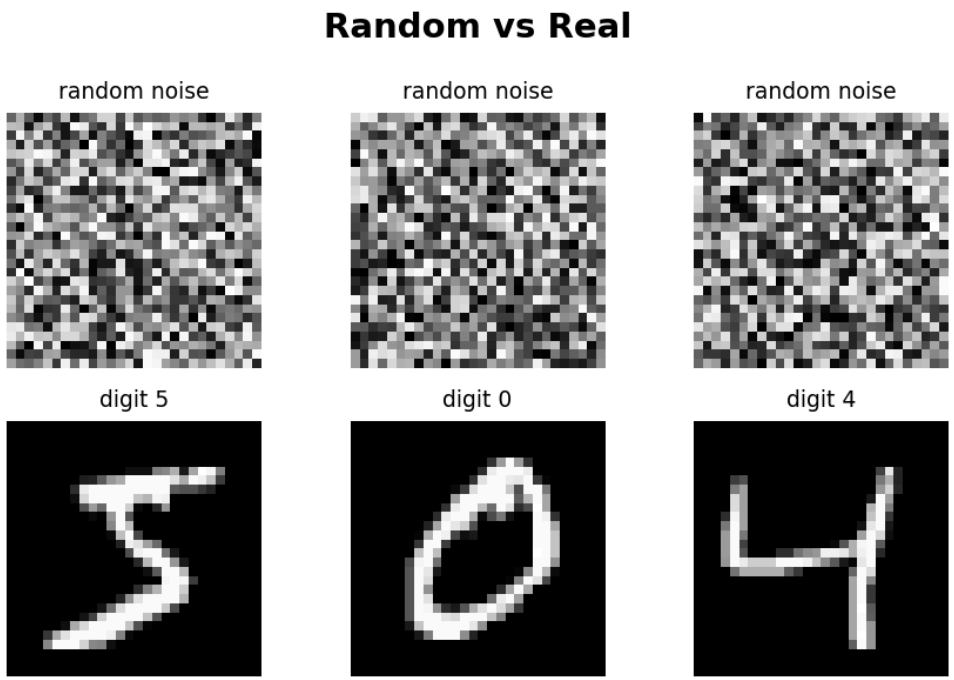
Quão grande é 10^{1887}?

Átomos no universo observável: $\sim 10^{80}$

10^{1887} não é "um número grande."

É um número tão grande que **todos os números grandes que já viram são zero por comparação.**

Quase todas são ruído



70.000 imagens reais num espaço de 10^{1887} possibilidades.

As imagens reais não estão dispersas aleatoriamente. Vivem numa superfície fina.

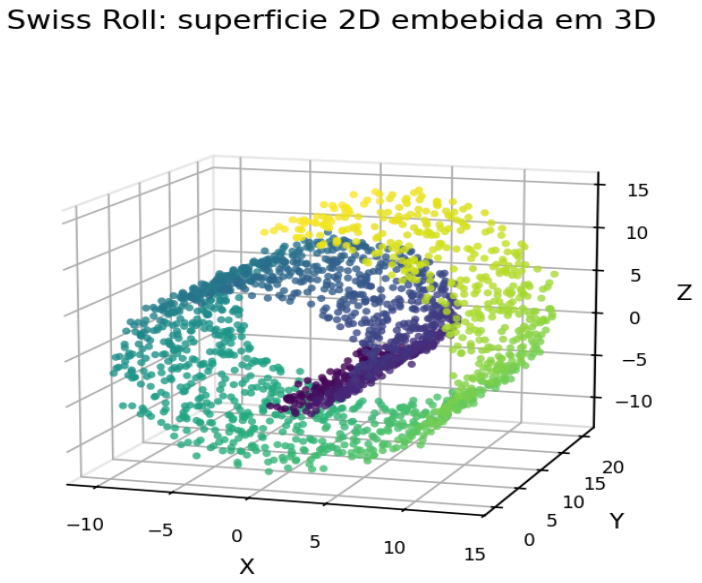
A manifold hypothesis

Imaginem uma folha de papel amarrotada numa bola.

A bola existe no espaço 3D — mas o papel em si é uma superfície 2D.

Os dados reais são o papel. O espaço 784-dimensional é a sala.

A manifold hypothesis: os dados reais vivem numa superfície de baixa dimensão embebida num espaço de alta dimensão.



Este é o desafio central

Aprender p(x) significa encontrar essa superfície.

Não procurar no espaço inteiro — isso é impossível.

Aprender onde os dados realmente vivem — a sua forma, estrutura, fronteiras.

Cada modelo generativo que vão estudar este semestre é uma resposta diferente a esta questão.

Podemos ver o manifold?

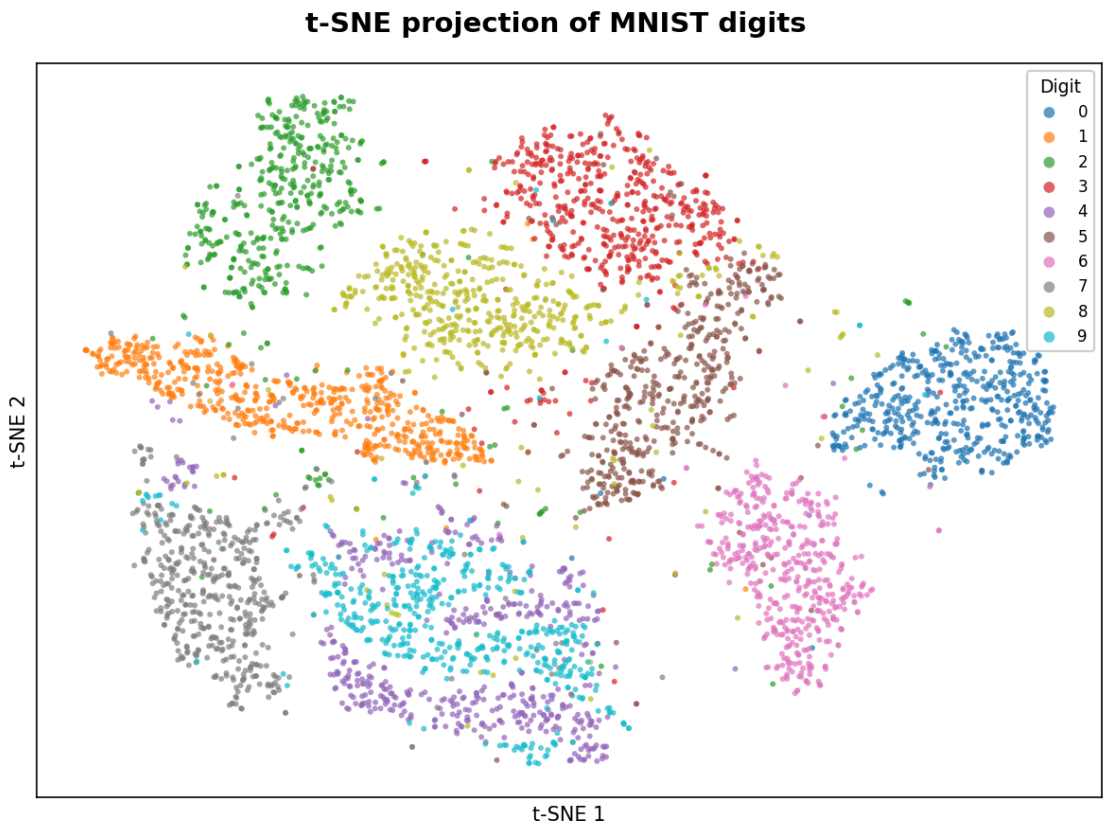
Não podemos visualizar 784 dimensões. Mas podemos projetar.

t-SNE (t-distributed Stochastic Neighbor Embedding):

- Recebe dados de alta dimensão
- Encontra um layout 2D que preserva **vizinhanças locais**
- Pontos próximos em 784D ficam próximos em 2D

É uma lente — imperfeita, mas reveladora.

O que o t-SNE nos mostra



Dígitos da mesma classe agrupam-se. Semelhantes (3/8, 4/9) ficam próximos.

Os dados **não** estão dispersos aleatoriamente — têm geometria.

Uma pergunta perigosa

Vimos a estrutura. Sabemos que os dígitos se agrupam.

E se fizermos a coisa mais simples possível?

1. Pegar em todas as imagens de um dígito (digamos, "3")
2. Calcular a média e covariância — ajustar uma Gaussiana
3. Fazer sampling de novos pontos dessa Gaussiana

Vamos obter 3s manuscritos realistas?

Não respondam já.

Vamos descobrir — mãos à obra.

Recapitulação antes do hands-on

Quatro ideias que vão testar nos próximos minutos:

1. **p(x) é a ideia central** de todo o generativo — estimar e fazer sampling
2. **10^{1887} imagens possíveis**, manifold minúsculo — os dados reais ocupam uma fracção ínfima
3. **Vamos tentar com uma Gaussiana** — será que funciona?
4. **A mesma ideia aplica-se a texto** — vamos ver o manifold das frases

Abram o terminal.

Hands-on: exploração do MNIST

Abrir o notebook:

```
jupyter notebook aula01-mnist.ipynb
```

(ou abrir directamente no VS Code com a extensão Jupyter)

O notebook tem **8 secções**. Corram célula a célula.

Antes de cada código: **prevejam o que vai acontecer**.

Se previrem correctamente — compreendem.
Se não — estão prestes a aprender algo.

Antes de começar — setup do ambiente

Criar um virtual environment para a UC (vão reutilizá-lo todas as semanas):

```
uv venv
.venv\Scripts\activate
uv pip install numpy matplotlib scikit-learn umap-learn jupyter
```

Depois abrir o notebook:

```
jupyter notebook aula01-mnist.ipynb
```

(Alternativa: abrir `aula01-mnist.ipynb` directamente no VS Code com a extensão Jupyter)

Se o download do MNIST falhar (sem internet), o notebook usa automaticamente um dataset mais pequeno.

Hands-on time

Então, o que aconteceu?

Os samples Gaussianos pareciam... errados.

Desfocados. Ruidosos. Não como dígitos manuscritos reais.

O modelo era matematicamente correcto. Estimámos uma distribuição real e fizemos sampling.

Então porquê o fracasso?

A Gaussiana não consegue capturar o manifold

Uma Gaussiana é um **blob** — simétrico, suave, elipsoidal.

Mas o manifold de 3s manuscritos não é um blob.

Tem curvas, ramificações, regiões finas.

Uma Gaussiana espalha massa de probabilidade **em todo o lado à volta da média** — incluindo o vasto espaço vazio onde nenhum dígito real existe.

A lição

Esta é a **tensão fundamental** da modelação generativa:

O modelo tem de ser poderoso o suficiente para capturar a **forma** do manifold — não apenas o seu centro e dispersão.

Uma Gaussiana em 784D? Demasiado simples. Não consegue aprender a geometria.

É por isso que precisamos de deep generative models.

Como fazemos melhor?

Uma ideia poderosa: **factorização autoregressiva**.

Já a experimentaram no script — o modelo bigram gera texto caracter a caracter, condicionando no anterior. O resultado já parecia português!

> Em vez de descrever a imagem toda de uma vez, descrever pixel a pixel:
> Qual deve ser o pixel 1? Dado o pixel 1, qual deve ser o pixel 2?
> É assim que o GPT gera texto — token a token.

$$p(x) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdots p(x_{784}|x_1, \ldots, x_{783})$$

O bigram condiciona em 1 caracter. O Transformer condiciona em TODOS os anteriores.

A ligação a tudo

Próxima semana: a **arquitectura Transformer**.

Uma máquina que implementa factorização autoregressiva com um mecanismo chamado **attention** — permitindo que cada token olhe para todos os tokens anteriores ao fazer a sua previsão.

A ideia que viram hoje — modelar p(x) decompondo em passos condicionais — é o fundamento matemático de cada LLM.

O que devem levar desta aula

1. IA generativa = modelar p(x), depois sampling
2. Os dados reais vivem num manifold de baixa dimensão num espaço enorme
3. Modelos simples (Gaussiana) não capturam a estrutura do manifold
4. A factorização autoregressiva é uma solução poderosa
5. **O Transformer é um motor para modelação autoregressiva**

Para a próxima semana

Leitura opcional:

- Karpathy, "The spelled-out intro to neural networks and backpropagation"

Na próxima aula: começamos a construir o **Transformer** — from scratch.